

Lab 3 – The Command Line: Shell, Pipes and Text Filters

Oct 5, 2026 · @Paty

Overview

By the end of this lab you will move fast in bash, predict what the shell does to a command line before it runs, redirect output where you want it, and answer real questions about data with one line of small filters joined by pipes.

Course: Operating Systems, FCIM / FAF, UTM, 2026-2027

Lab 3: The Command Line (Track A: the real OS, from the outside, Linux)

Duration: 90 minutes

Golden rule: before you press Enter on any command marked **PREDICT**, write your prediction in your notes. Then run it. The mismatches are where you learn.

General requirements (for every lab)

1. Word report (.docx) following the UTM structure

- UTM title page: university, faculty, department; “Report on laboratory work no. 3”, course, topic; “Done by: st. gr. ____, Name Surname”, “Checked by: ____”; Chişinău, 2026.
- Content: purpose of the work, the work done part by part (predictions, outputs, answers to every **Observe** box and to Q1-Q8), **conclusions**.
- Every screenshot is centered, with a centered caption below it: **Figure 1.1 – Name of the figure** (first number = the part, second = the figure number within that part).
- File name: Lab3_NameSurname_Group.docx.

2. Git account for your labs

- Create a [GitHub](#) account and a repository named **os-2026-NameSurname**; send the link to your teacher.
- Each lab gets its own folder (here **lab3/**: the .docx report, report.sh, oneliners.sh), uploaded with:

```
git add lab3 && git commit -m "Lab 3" && git push
```

Before you start (~5 min)

1. Open a terminal on your Linux laptop. Keep a notes file (e.g. notes.md) open beside it for predictions and outputs; you will build the Word report from it.
2. If your laptop is not prepared yet, run the course setup script once (it also prepares xv6 for Lab 4):

```
bash os2026-setup.sh
```

3. Generate the data set for Part 4 (an artificial web access log, the same for everyone). Copy the block exactly:

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $(i % 5)) -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log ; wc -l access.log
```

Check: the last command must print **500 access.log**. If not, generate the file again.

Part 1. Move fast, find help (~15 min)

What to do:

1. Type `ls /usr/sh` and press **Tab twice**. Then type `cd /et` and press **Tab once**. Note what Tab did in each case.
2. Run any three different commands. Then press **Ctrl+R**, type a few letters of the first one and press Enter. Note what happened.
3. Find out whether a command is built-in or external, and where the shell looks for it:

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'          # one directory per line
nosuchprog ; echo "exit code: $?"
```

4. Find help without a browser:

```
man ls                # inside: /sort then n for next, q to quit
ls --help | head
man -k "list directory"
help cd
```

Observe (answer in the report):

1. Which of cd, ls, echo are built-ins, and why MUST cd be one?
2. How many directories are in your PATH, and in which one is ls?
3. What exit code does a command that is not found return?
4. Which option of ls sorts by modification time? (find it in man ls)

Minimum screenshots: Figure 1.1 – Tab completion; Figure 1.2 – Output of type and which; Figure 1.3 – The directories in PATH; Figure 1.4 – Exit code of a missing command.

Part 2. What the shell does to your line (~15 min)

Setup: create a test directory with a few files:

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

PREDICT: for each line below, FIRST write down what you think it will print, then run it and note the real output.

```
echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
ls alfa beta.txt
ls "alfa beta.txt"
echo '$HOME' "$HOME" \ $HOME
echo {1..5} ; echo $((7 * 6))
echo "today is $(date +%A), I am $(whoami)"
touch log{1..3}.txt && ls log*
```

In the report, present the results as a table:

Command	My prediction	Real output	Explanation (if different)
echo *.c			
...			

Observe (answer in the report):

1. For each line: your prediction, the real output, and one sentence if they differ (the table above).
2. echo knows nothing about files, yet echo *.c printed file names. Who produced them?
3. Why did ls alfa beta.txt fail? Give two different ways to fix it.

4. What is the difference between single and double quotes?

Note: ls may show the file with a space as 'alfa beta.txt' (in quotes). Explain why it does that.

Minimum screenshots: Figure 2.1 – Files created in the play directory; Figure 2.2 – Globbing results; Figure 2.3 – The ls error on a name with a space, and the fix; Figure 2.4 – Single quotes, double quotes and backslash.

Part 3. Redirection, chaining and jobs (~15 min)

What to do: run the commands below one by one and note what each prints.

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
sleep 100 &
jobs ; kill %1 ; jobs
```

For question 2 below, run and compare these two versions:

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

Observe (answer in the report):

1. Which file got the error message, and which got the listing?
2. What is the difference between `> all.txt 2>&1` and `2>&1 > all.txt`? Try both and explain why the result differs.
3. Why does `wc -l < access.log` print only a number, while `wc -l access.log` also prints the file name?
4. Explain each operator in one sentence: `;` `&&` `||`.

Minimum screenshots: Figure 3.1 – stdout and stderr in separate files; Figure 3.2 – Comparing the order of redirections; Figure 3.3 – The `;` `&&` `||` operators; Figure 3.4 – Running a background job and stopping it.

Part 4. Answer questions with one-liners (~25 min)

Each line of access.log is: date, time, user, result (OK or FAIL), page.

What to do: answer every question with ONE command line (pipes allowed). Save all lines in a file **oneliners.sh**, one per question, with the question as a comment above it. Useful tools: grep, cut, sort, uniq, wc, head, tail, tr.

- **Q1.** How many requests failed?
- **Q2.** How many different pages were requested, and which?
- **Q3.** How many requests did each page get?
- **Q4.** Which user or users have the most failed requests, and how many?
- **Q5.** How many requests did user3 make, and how many of them failed?
- **Q6.** Print the last 3 failed requests, showing only time and user.
- **Q7.** Which login shells appear in /etc/passwd, and how many accounts use each? (field 7, separator :)
- **Q8.** PREDICT, then explain: why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by exactly one?

A pattern to start from (count something per user):

```
grep OK access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

Format of oneliners.sh:

```
# Q1. How many requests failed?
<your command here>

# Q2. How many different pages were requested, and which?
<your command here>
```

Check yourself: the log has 500 lines. Your Q3 counts must add up to 500, and the per-user failure counts behind Q4 must add up to your Q1. If not, the pipeline is filtering the wrong thing. The file must run with `bash oneliners.sh` inside ~/os-lab3.

Bonus (+1): with `grep -E`, print only the lines where the minute ends in 0 (10:00, 10:10, ...) and the request failed. How many are there?

Minimum screenshots: one figure per question (Figure 4.1 – Number of failed requests, ... Figure 4.8), plus Figure 4.9 – Full run of oneliners.sh.

Part 5. Your first script (~10 min)

What to do: turn your best one-liners into a program. Create **report.sh** in `~/os-lab3` with the content below:

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
# TODO: for each of user0 ... user6, print "userN: X failures"
```

Complete the TODO: a loop that prints “userN: X failures” for every user from user0 to user6. Then run:

```
chmod +x report.sh
./report.sh access.log
./report.sh nope.log ; echo "exit code: $?"
```

Observe (answer in the report):

1. What does the first line `#!/bin/bash` do, and what happens if you run `./report.sh` before `chmod +x`?
2. Why must you type `./report.sh` and not just `report.sh`?
3. Why does the script write the error with `>&2` and end with `exit 1`?

Bonus (+1), run it with cron: add a line to your crontab (`crontab -e`) that runs `report.sh` every minute and appends its output to `~/os-lab3/report.log` (use full paths and `>> ... 2>&1`). Wait two minutes, show the log, then remove the line (`crontab -e` , or `crontab -r` to delete the whole crontab). Put your crontab line and the log in the report.

Minimum screenshots: Figure 5.1 – Contents of `report.sh`; Figure 5.2 – Running the script on `access.log`; Figure 5.3 – Error handling for a missing file.

Deliverables and grading

In the **lab3/** folder of your Git repository: the report `Lab3_NameSurname_Group.docx`, `report.sh` (TODO done) and `oneliners.sh` (runs with `bash oneliners.sh` inside `~/os-lab3`). When grading, `access.log` is regenerated and your one-liners are run on it.